

1. Hello.c

```
#include<stdio.h>
```

```
int main()
{
    printf("Hello Linux\n");
    return 0;
}
```

Compile: gcc hello.c -o hello

./hello

OUTPUT:

```
adminrishik@LAPTOP-BSDK1KRC:~$ nano hello1.c
adminrishik@LAPTOP-BSDK1KRC:~$ gcc hello1.c -o hello1
adminrishik@LAPTOP-BSDK1KRC:~$ ./hello1
Hello Linux
```

2. Read User Name

```
#include<stdio.h>
```

```
int main()
{
    char name[30];

    printf("Enter your name: ");
    scanf("%s",name);

    printf("Welcome %s\n",name);

    return 0;
}
```

OUTPUT:

```
adminrishik@LAPTOP-BSDK1KRC:~$ nano hello2.c
adminrishik@LAPTOP-BSDK1KRC:~$ gcc hello2.c -o hello2
adminrishik@LAPTOP-BSDK1KRC:~$ ./hello2
Enter your name: Rishik Ramagiri
WelcomeRishik
```

3. Display Process ID

```
#include<stdio.h>
```

```
#include<unistd.h>
```

```
int main()
{
```

```

printf("Process ID = %d\n",getpid());

return 0;
}

```

OUTPUT:

```

adminrishik@LAPTOP-BSDK1KRC:~$ nano hello3.c
adminrishik@LAPTOP-BSDK1KRC:~$ gcc hello3.c -o hello3
adminrishik@LAPTOP-BSDK1KRC:~$ ./hello3
Process ID = 673

```

4. Fork()

```

#include<stdio.h>
#include<unistd.h>

int main()
{
    fork();

    printf("Hello\n");

    return 0;
}

```

OUTPUT:

```

adminrishik@LAPTOP-BSDK1KRC:~$ nano hello4.c
adminrishik@LAPTOP-BSDK1KRC:~$ gcc hello4.c -o hello4
adminrishik@LAPTOP-BSDK1KRC:~$ ./hello4
Hello
Hello

```

5. Parent and Child

```

#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("I am Child Process\n");
    }
    else

```

```

    {
        printf("I am Parent Process\n");
    }

    return 0;
}

```

OUTPUT:

```

1 adminrishik@LAPTOP-BSDK1KRC:~$ nano hello5.c
adminrishik@LAPTOP-BSDK1KRC:~$ gcc hello5.c -o hello5
adminrishik@LAPTOP-BSDK1KRC:~$ ./hello5
I am Parent Process
I am Child Process

```

6. Display parent and Child PID

```

#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("Child PID = %d\n",getpid());
        printf("Parent PID = %d\n",getppid());
    }
    else
    {
        printf("Parent PID = %d\n",getpid());
    }

    return 0;
}

```

OUTPUT:

```

adminrishik@LAPTOP-BSDK1KRC:~$ nano hello6.c
adminrishik@LAPTOP-BSDK1KRC:~$ gcc hello6.c -o hello6
adminrishik@LAPTOP-BSDK1KRC:~$ ./hello6
Parent PID = 798
Child PID = 799
Parent PID = 798

```

7. Wait()

```

#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        printf("Child Running\n");
    }
    else
    {
        wait(NULL);

        printf("Parent Running After Child\n");
    }

    return 0;
}

```

OUTPUT:

```

adminrishik@LAPTOP-BSDK1KRC:~$ nano hello7.c
adminrishik@LAPTOP-BSDK1KRC:~$ gcc hello7.c -o hello7
adminrishik@LAPTOP-BSDK1KRC:~$ ./hello7
Child Running
Parent Running After Child

```

8. Wait()

```

#include<stdio.h>
#include<unistd.h>

int main()
{
    printf("Before exec()\n");

    execlp("ls","ls",NULL);
}

```

```

printf("After exec()\n");

return 0;
}

```

OUTPUT:

```

1 adminrishik@LAPTOP-BSDK1KRC:~$ nano hello8.c
adminrishik@LAPTOP-BSDK1KRC:~$ gcc hello8.c -o hello8
adminrishik@LAPTOP-BSDK1KRC:~$ ./hello8
Before exec()
example.txt  hello.c  hello1.c  hello2.c  hello3.c  hello4.c  hello5.c  hello6.c  hello7.c  hello8.c  test
hello        hello1  hello2  hello3  hello4  hello5  hello6  hello7  hello8  s25ampleProgram.c

```

9. fork() + exec()

```

#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>

int main()
{
    pid_t pid;

    pid=fork();

    if(pid==0)
    {
        execlp("date","date",NULL);
    }
    else
    {
        wait(NULL);

        printf("Child Completed\n");
    }

    return 0;
}

```

OUTPUT:

```

adminrishik@LAPTOP-BSDK1KRC:~$ nano hello9.c
adminrishik@LAPTOP-BSDK1KRC:~$ gcc hello9.c -o hello9
adminrishik@LAPTOP-BSDK1KRC:~$ ./hello9
Thu Aug  6 04:55:32 UTC 2026
Child Completed

```